

system-design-interview / 4주차 / 6장 · 7장

키 - 값 저장소 설계 & 분산 시스템용 유일 ID 생성기

6장 · 7장 | 대규모 시스템 설계 스터디 4주차

송승환

2026.09

발표 구성

요약보다 — 제가 막혔던 개념 + AWS 연결 위주로 구성

01 책 요약은 짧게 진행

다들 읽고 오셨으니 전체 흐름만 한 장으로

02 헛갈리는 개념 집중

제가 읽으며 막혔던 지점 = 아마 여러분들도 궁금했을 지점

03 AWS 연결

SAA 준비하며 배운 서비스로 책 개념을 엮어봄

=> 책 요약은 흐름만 짚고 , 막히는 개념과 AWS 연결에 시간을 씀

6 장 한눈에 보기

단일 서버에서 출발해 한계를 만나고, 그 한계를 푸는 순서

start.md

단일 서버 KV 저장소

키 - 값 쌍 전부를 메모리 해시 테이블에 저장

limits.md

한계 메모리 용량 · 단일 장애점

→ 여러 서버로 분산해야 한다

↓ 분산하면 새로 풀어야 할 네 가지

데이터 파티션

어디에 나눠 담나
(안정 해시)

데이터 다중화

몇 벌 복사하나
(N 개 사본)

일관성

어떤 값을 최신으로 보나
(CAP · W/R · 벡터 시계)

장애 처리

죽으면 어떻게 하나
(가십 · 머클 트리)

=> 오늘은 이 네 가지에서 막히기 쉬운 지점만 골라서 봄

Q1. 왜 키 - 값 저장소가 필요한가

DB 는 한 종류가 아님 — 키 - 값은 그중 가장 단순한 DB

kv-store.txt

```
"user:123"    →  " 송승환 "  
"session:abc" →  { 로그인 정보 }  
"cart:u123"   →  [ 상품 1, 상품 2]
```

key 주면 value 뵈음 . JOIN 없고 테이블 없고 SQL 없음

이유

왜

빠름

파싱 · 실행 계획 없이 해시맵 O(1)

샤딩이 쉬움

JOIN · 트랜잭션이 없어 쪼개기 쉬움

단순함

단순 조회엔 RDB 가 오버스펙

chapter-04.md

우리는 이미 KV 저장소를 써봤음 . 4 장 Rate Limiter 의 카운터를 Redis 에 저장한 것 .

user123:count → 5

Redis 가 바로 키 - 값 저장소이고 , 6 장은

그것을 어떻게 만드느지를 다룸 .

=> KV 는 데이터 모델이 가장 단순해서 , 분산의 진짜 어려운 부분에만 집중할 수 있음

Q2. RDB 의 JOIN 과 트랜잭션이란

KV 에는 없는 두 가지 — 그리고 그것이 분산을 어렵게 만드는 이유

join.sql

JOIN — 흩어진 테이블을 엮어 조회

[users]		[orders]	
id	name	id	user_id
1	송승환	→ 101	1

```
SELECT u.name, o. 상품
FROM users u JOIN orders o
      ON u.id = o.user_id;
```

transaction.log

트랜잭션 — 다 되거나, 다 안 되거나

BEGIN

① A 잔액 -10000

② B 잔액 +10000

COMMIT 둘 다 성공해야 확정

ROLLBACK 하나라도 실패 → 통째로 취소

ACID: 원자성 · 일관성 · 격리성 · 지속성

why-it-matters.md

데이터를 여러 서버에 쪼개면 (샤딩) JOIN 은 서버를 넘나들어야 하고, 트랜잭션은 여러 서버에 걸쳐 보장해야 함

=> RDB 의 장점이 데이터를 쪼개는 순간 독이 됨 — KV 는 이 둘이 없어 쪼개도 앓을 게 없음

Q3. 복제와 파티션은 다르다

'DB 를 여러 벌 둔다'에는 성격이 완전히 다른 두 방식이 있음

replication.txt

복제 (Replication) — 똑같이 복사

DB1 [전체] → DB2 [전체] → DB3 [전체]
원본 복사본 복사본

각 DB 내용이 같음 · 장애 대비 + 읽기 분산

partition.txt

파티션 (Sharding) — 쪼개서 나눠 담기

DB1 [1~100] DB2 [101~200] DB3 [201~]
서로 다른 조각

각 DB 내용이 다름 · 저장 용량 확장 (5 장 안정 해시)

in-practice.md

AWS ElastiCache 엔진 선택에 이 구도가 그대로 나타남 — **Redis 는 둘 다 가능** (클러스터 모드를 켜면 샤딩한 각 샤드를 다시 복제),
Memcached 는 샤딩만 하고 복제 · 고가용성이 없음

=> 복제와 파티션은 함께 씀 — 쪼개 각 조각을 다시 복제 . RDB 는 파티션이 어려워 복제에 의존

Q4. " 메모리에 저장한다 " 의 그 메모리는 ?

KV 저장소는 애플리케이션 서버가 아니라 DB 그 자체

architecture.txt

[EC2 서버] —→ [KV 저장소]
 앱 로직 = DB 그 자체
 (Redis 같은)

machine.txt

[DB 머신]
 └─ RAM 빠름 · 꺼지면 날아감
 └─ 디스크 느림 · 꺼져도 보존

DB	주 저장 위치	특징
MySQL	디스크 (+ RAM 캐시)	안 날아감, 대신 상대적으로 느림
Redis	RAM (+ 디스크 백업)	매우 빠름, 대신 비싸고 용량 한계

=> DB 도 프로그램이라 RAM 을 씀 . RAM 은 빠르지만 휘발성 · 용량 한계 → 그래서 분산이 필요해짐

Q5. CAP — 가용성과 파티션 감내는 왜 별개인가

셋이 동급으로 나열되지만, 실제로는 P가 먼저 터지고 A냐 C냐를 고르는 구조

P 파티션 감내

네트워크가 끊기는 사고를 견디는 능력

A 가용성

그 사고 중에도 응답을 주는가

C 일관성

그 사고 중에도 정확한 값만 주는가

partition-occurred.log

네트워크 단절 발생

ERR s0 --x-- s1

A> 응답 반환 (옛 값일 수 있음)

C> 응답 거부 (정확성 우선)

=> 분산이면 P는 필수 → CP or AP

consistency-models.md

강한 일관성 항상 최신 값

약한 일관성 최신 보장 없음

결과적 일관성 시간이 지나면 수렴

=> P는 ' 정전 사태 ', A는 ' 정전에도 촛불 켜고 장사 ', C는 ' 정확히 못 할 바엔 문 닫음 '

Q6. 데이터 다중화와 정족수 ($N \cdot W \cdot R$)

다중화 = KV 의 복제 — 해시 링에서 시계방향으로 사본을 둬

replication-rule.txt

key Hashing → 링 위 한 지점
→ 시계방향으로 서버 N 개 선택
→ 단, 같은 물리 서버는 건너뛰

가상 노드가 겹치면 한 대 죽을 때
사본이 같이 날아가기 때문

기호

의미

N

사본 개수 — 링에 저장한 그 개수

W

쓰기 정족수 — W 개 서버가 ack 주면 쓰기 성공

R

읽기 정족수 — R 개 서버가 응답하면 읽기 성공

write-flow.log

$N=3$, $W=2$ 쓰기 → 중재자가 $s1$, $s2$, $s3$ 에 값을 뿌림
 $s1$ ack $s2$ ack $s3$... → ack 2 개 도착 = W 충족 → 쓰기 성공 확정

=> 사본은 물리적으로 다른 서버 N 개에 두고, ack 가 $W \cdot R$ 개 모이면 성공으로 처리함

Q6-2. 왜 $W+R>N$ 이면 강한 일관성인가

쓰는 집합과 읽은 집합이 반드시 겹치도록 만드는 조건

quorum-overlap.txt

$N=3, W=2, R=2$ ($W+R=4 > N=3$)

쓰기 [s1] [s2] s3

읽기 s1 [s2] [s3]

↑ 최소 1 대는 반드시 겹침
(비둘기집 원리)

겹치는 서버가 최신값을 갖고 있음

→ 읽을 때 최신값이 무조건 끼어 있음

→ 그것을 골라 주면 됨

= 강한 일관성 보장

설정	성향	결과
W, R 크게 ($W+R>N$)	CP	강한 일관성 우선 · ack 를 더 기다려 느낌
W, R 작게 ($W=R=1$)	AP	빠르고 가용성 높음 · 최신 보장 안 됨

=> W와 R 숫자를 조절하는 것이 곧 CP/AP 를 정하는 것

Aurora — 책의 정족수가 실제로 돌아가는 곳

데이터를 3 개 AZ 에 6 벌 저장하고, 쓰기는 4 개 · 읽기는 3 개의 응답을 기다림

aurora-storage.txt

AZ 1 AZ 2 AZ 3
[■] [■] [■] [■] [■] [■] 사본 6 벌

쓰기 $W=4$ 6 개 중 4 개의 ack 를 기다림

읽기 $R=3$ 6 개 중 3 개의 응답을 기다림

$$N = 6 \quad W = 4 \quad R = 3$$

$$W + R = 7 > N = 6$$

쓴 4 개와 읽은 3 개는 반드시 겹침 ($4+3 > 6$)

= 강한 일관성 보장

note.md

Aurora 는 노드끼리 P2P 로 복제하며 손상된 사본을 스스로 복구함

→ 6 장의 다중화 · 정족수 · 장애 복구가 한 서비스에 다 들어 있는 셈 (RDS 보다 20% 비싸지만, 알아서 다 해줌)

=> 책의 $W+R>N$ 은 이론만이 아니라, 지금 이 순간 Aurora 가 이 숫자로 돌아가고 있음

잠깐 — 로드밸런서 (LB) 란

다음 장에서 중재자와 비교하려면 이것부터 알아야 함

load-balancer.txt

```
┌──→ 서버 1
클라 → [LB] → 서버 2
└──→ 서버 3
```

요청 하나를 셋 중 한 대에 넘김

트래픽 분배

요청을 서버들에 나눠 보냄

죽은 서버 제외

Health Check 로 확인해 뺌

서버 증감 흡수

서버가 늘고 줄어도 클라는 모름

distribution.md

Round Robin 순서대로 돌아가며 나눔 — 대부분의 기본값

Least Connections 처리 중인 요청이 가장 적은 서버로 — 요청마다 처리 시간이 크게 다를 때 유리

AWS 에서는 ELB 가 이 역할을 하고 , ASG 와 함께 서버 대수를 자동 조절함 . ALB 기본값은 Round Robin

=> 그럼 6 장의 ' 중재자 ' 는 이것과 뭐가 다를까요 ?

Q7. 중재자는 로드밸런서와 뭐가 다른가

요청을 받는다는 점은 닮았지만, 하는 일이 다름

load-balancer.txt

로드밸런서 — 한 대에 떠넘김

요청 → [LB] → 서버 1

별도 계층 (AWS ELB)

한 대를 골라 넘기면 끝

coordinator.txt

중재자 — 뿌리고 정족수를 셈

요청 → [중재자] → 사본 1
→ 사본 2
→ 사본 3

ack W개 모이면 성공 처리

how-is-it-chosen.md

중재자는 어떻게 정해지나

→ 링 위 노드 중 하나가 요청마다 임시로 맡는 **역할 (role)**. 보통 요청을 처음 받은 노드가 됨.

별도 장비도, 고정 서버도 아님. **자기가 사본이 아니어도** 사본들에 뿌리고 정족수를 세는 조율만 할 수 있음.

=> LB 는 한 대에 떠넘기고, 중재자는 사본 전체에 뿌린 뒤 정족수를 셈

Q8. 데이터 버저닝 — 이전 버전 vs 충돌

Vector Clock · $D([s0, 1], [s1, 2]) =$ 서버 s0 이 1 번, s1 이 2 번 수정했다는 이력 도장

rule-1: 이전 버전

$D([s0, 1], [s1, 1]) \leftarrow X$
 $D([s0, 1], [s1, 2]) \leftarrow Y$

s0: $1 \leq 1$ s1: $1 \leq 2$ → X는 Y의 조상

rule-2: 충돌

$D([s0, 1], [s1, 2]) \leftarrow Y$
 $D([s0, 2], [s1, 1]) \leftarrow Z$

s0: Z 우세 s1: Y 우세 → 충돌

why-conflict-happens.log

원본 $D([s0, 1], [s1, 1]) \rightarrow$ s0 --x-- s1 네트워크 단절

s1 쪽 : 클라 A가 수정 $\rightarrow D([s0, 1], [s1, 2])$ s0 쪽 : 클라 B가 수정 $\rightarrow D([s0, 2], [s1, 1])$

복구 후 두 버전이 만남 $\rightarrow$ 둘 다 원본에서 갈라져 각자 발전 $\rightarrow$ 누가 최신인지 못 정함

=> Git 브랜치 충돌과 같은 원리 . 벡터 시계는 충돌 감지기이지 해결사가 아님 — 해결은 앱 로직 몫

장애 처리 — 감지부터 복구까지

책은 장애를 세 종류로 나누고, 각각 다른 방법으로 다룸

01 장애 감지 가십 (Gossip) 프로토콜

노드끼리 heartbeat 를 주고받으며 상태를 퍼뜨림 . 여러 노드가 같은 노드를 죽었다고 보고하면 확정

02 일시적 장애 느슨한 정족수 (Sloppy Quorum) + Hinted Handoff

죽은 노드 대신 다른 노드가 요청을 임시로 받아두고, 되살아나면 넘겨줌

03 영구 장애 Anti-Entropy + 머클 트리 (Merkle Tree)

사본끼리 데이터를 비교해 차이 나는 부분만 동기화

=> 감지는 소문으로 , 잠깐 죽으면 대신 받아줬다가 , 영영 죽으면 사본끼리 맞춰봄

가십 프로토콜은 실제로 어떻게 도는가

중앙 감시자 없이, 노드끼리 서로의 상태를 퍼뜨림

gossip-cycle.txt

- (1) 내 박동 카운터를 +1
" 나 살아있다 "
- (2) 무작위 노드 몇 개에
내가 아는 카운터 목록 전체를 보냄
- (3) 받은 노드는 더 큰 값만 채택
- (4) 한참 안 변하면 장애로 판단

membership-list.txt

	b 목록	a 가 보낸	결과
a	5	< 9	9 로 갱신
b	12	> 11	무시
c	6	< 7	7 로 갱신

b 는 c 와 직접 통신한 적이 없는데
a 를 거쳐 c 소식을 들음

note.md

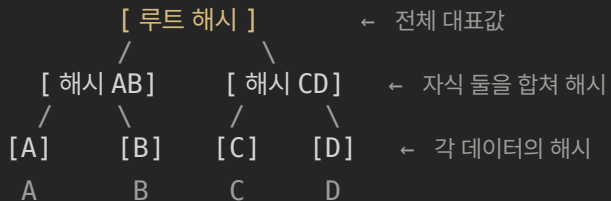
카운터는 주인만 올림 — 남이 대신 올려주면 그 노드가 죽어도 영영 살아있는 것으로 보임 . 멤버십 목록에는 마지막 갱신 시각도 있지만 전송하지는 않음 (내 시계 기준이라 나에게만 의미가 있고 , 장애 판정에만 씀)

=> 시각이 아니라 카운터를 쓰는 이유는 서버마다 시계가 다르기 때문 — 커지기만 하면 비교가 됨

Q9. 머클 트리 — 차이 나는 데이터만 골라내기

Merkle Tree · 영구 장애 복구 시, 100 만 개를 다 비교하지 않고 다른 것만 찾는 방법

merkle-tree.txt



비교 방법

- ① 루트 해시가 같다 → 전체 동일 . 끝
- ② 다르다 → 자식들 비교
- ③ 같은 가지는 통째로 스킵
- ④ 다른 가지만 파고들

→ 100 만 개 중 3 개만 달라도

그 3 개만 찾아 전송

analogy.md

책 두 권이 같은지 확인할 때 첫 장부터 다 읽지 않고 "목차 같아? → 몇 장이 달라? → 그 장 안에서 몇 페이지?" 로 좁혀가는 것 .

비트코인이 블록 안의 거래들을 묶어 검증하는 그 원리를 , 여기서 서버 간 데이터 차이 찾기에 씀 .

=> 차이 나는 데이터만 골라내 전송 — 비트코인이 거래를 검증하는 그 원리

쓰기와 읽기는 내부에서 이렇게 흐른다

메모리에 먼저 쓰고 , 나중에 디스크로 내리는 구조

write-path.txt

- (1) commit log 에 먼저 기록
장애로 꺼져도 복구 가능
- (2) memtable(메모리) 에 저장
- (3) 가득 차면 SSTable 로
디스크에 flush

read-path.txt

- (1) memtable 에 있나 확인
있으면 바로 반환
- (2) 없으면 Bloom Filter 로
후보 SSTable 을 추린 뒤 조회

bloom-filter.md

Bloom Filter 는 " 확실히 없다 " 를 빠르게 판단하는 자료구조임 . SSTable 을 전부 뒤지지 않고 후보를 좁혀 , 디스크 조회 횟수를 크게 줄여줌 .

=> 쓰기는 메모리에 빠르게 , 읽기는 Bloom Filter 로 디스크 접근을 줄임

6 장 개념 ↔ AWS 서비스 매핑

SAA 준비하며 배운 서비스로 책 개념을 엮어보면

6 장 개념	AWS 서비스	연결 포인트
분산 키 - 값 저장소	ElastiCache (Redis)	Redis 자체가 분산 인메모리 KV 저장소
데이터 파티션	Redis Cluster	고정 슬롯을 노드에 나눠 배정 — 가상 노드와 같은 발상
데이터 다중화	RDS Multi-AZ / Read Replica	원본을 여러 벌 복사해 장애 · 읽기 대비
일관성 모델	Multi-AZ vs Read Replica	동기 복제 = 강한 일관성 / 비동기 = 결과적
장애 감지	ElastiCache Redis Cluster	노드끼리 서로의 상태를 주고받음 = 가십 프로토콜
고가용성	ELB + ASG, 다중 AZ	한 대 죽어도 서비스 유지

=> 6 장의 Dynamo 논문이 곧 AWS DynamoDB의 조상 — 강의 뒤 서버리스 파트에서 배울 예정

AWS 연결 ① RDS 복제 = 6 장 일관성 모델

책에서 읽은 두 일관성 모델이 RDS 에 그대로 있음

방식	복제	일관성	6 장 대응
Multi-AZ	동기 (synchronous)	강한 일관성	$W+R > N$
Read Replica	비동기 (asynchronous)	결과적 일관성	eventual consistency

주의 Multi-AZ 의 대기 인스턴스는 읽기를 받지 않음 — 확장용이 아니라 순수 장애 복구 (DR) 용

```
replication-lag.log
```

t0 원본 DB 에 write → " 송승환 "

t1 Read Replica 에서 read → 아직 옛 값 (복제 지연 replication lag)

t2 복제 완료 → 이제 최신값 = 결과적 일관성

=> 방금 쓴 데이터가 Read Replica 에서 아직 안 보이는 것 — 책이 말한 결과적 일관성의 실제 사례

번외 : 실무에서는 어떻게 쓰나

책 내용은 아니지만 , 6 장의 복제 · 파티션이 실제로 어떤 순서로 쓰이는지

role-split.md

RDS 나 KV 나 → 섞어 씀

유저 · 주문 · 결제 → RDS
JOIN · 트랜잭션 , 정합성 중요

캐시 · 세션 · 랭킹 → Redis
빠른 조회 , 좀 틀려도 안 죽음

scaling-order.md

RDS 확장 순서 — 샤딩은 최후

- ① 복제 Multi-AZ · Read Replica
- ② 수직 확장 인스턴스 사양 업
- ③ 파티션 정말 안 될 때만 샤딩
JOIN · 트랜잭션이 깨져 앱이 복잡해짐

why.md

대부분의 서비스는 읽기가 쓰기보다 훨씬 많아 Read Replica 몇 대로 충분함 . "샤딩은 최대한 미룬다" 가 실무 기본값 .
그래서 초대규모 서비스는 아예 처음부터 샤딩이 쉬운 NoSQL/KV 를 메인으로 쓰기도 함 .

=> 복제로 버티다 정 안 될 때만 샤딩 — 메인은 RDS , 빠르게 자주 쓰는 건 Redis

7장 . 유일 ID 생성 — 네 가지 방식

분산 환경에서 중복 없는 ID 를 어떻게 만들 것인가

방식	원리	장점	단점
다중 마스터 복제	auto_increment 보폭을 서버 수만큼	기존 DB 그대로	시간 정렬 깨짐, 서버 추가 시 곤란
UUID	128 비트 난수	조율 불필요, 충돌 사실상 없음	숫자 아님, 길고, 시간 정렬 안 됨
티켓 서버	중앙 DB 가 auto_increment 로 발급	숫자 ID, 구현 단순	SPOF — 티켓 서버가 죽으면 전체 정지
Snowflake	64 비트를 의미 단위로 쪼갬	시간 정렬 O, 조율 없이 생성 O	시계 동기화 문제를 따로 고려해야 함

=> RDS 의 auto_increment = 책의 **티켓 서버** 방식 — 중앙 DB 가 ID 를 발급하므로 SPOF 위험이 그대로 따라옴

7 장 . Snowflake — 64 비트를 쪼개 쓰기

각 비트에 의미를 부여해 조울 없이도 시간순 정렬되는 ID 를 만듦

1 bit 부호	41 bit 타임스탬프	5 bit 데이터센터	5 bit 서버	12 bit 일련번호
-------------	-----------------	----------------	-------------	----------------

0 고정

기준 시각 이후 밀리초

최대 32 개

DC 당 32 대

같은 ms 내 4096 개

총 64 bit (그림의 폭은 가독성을 위한 것으로 실제 비트 비율과 다름)

why-it-works.md

상위 비트가 시간

- 숫자 크기순 = 시간순 정렬
- 중앙 조울 없이 각 서버가 생성

aws-note.md

정렬 키에 시간 정렬 ID 를 쓰면

시계열 조화가 편해짐 — Snowflake 가 시간 정렬을 중요하게 본
이유

=> 상위 비트가 시간이라 숫자 크기순이 곧 시간순 — 조울 없이 각 서버가 생성

Snowflake 의 남은 과제 — 서버 시계는 믿을 수 있나

상위 비트가 시간이라는 설계는, 그 시간을 믿을 수 있을 때만 성립함

problem.txt

상위 41 비트 = 타임스탬프
→ 숫자 크기순 = 시간순 정렬

그런데 서버마다 시계가 다르면 ?

s1 의 12:00:01 과
s2 의 12:00:01 이 다른 순간
→ 정렬이 어긋남

ntp.txt

NTP (Network Time Protocol)
UDP 123 · 서버 간 시계를 맞춤

Stratum 0 원자시계 · GPS 수신기
Stratum 1 0 에 직접 연결된 서버
Stratum 2 1 과 동기화된 서버
Stratum 3 보통 우리가 쓰는 것

caveat.md

NTP 도 만능은 아님 — 보정 과정에서 시계가 뒤로 갈 수 있고, 그러면 이미 발급한 ID 와 겹칠 위험이 생김.
그래서 실제 구현은 시계가 뒤로 가면 ID 발급을 멈추고, 시계가 따라올 때까지 기다림.

=> 시간에 기대는 설계는 시계를 믿을 수 있어야 성립함 — 그 믿음을 만들어주는 것이 NTP

정리 그리고 토의

6 장 KV 는 JOIN·트랜잭션을 포기한 대신 확장을 얻었고 , 그래서 파티션·복제·일관성·장애 처리를 직접 풀어야 함

일관성 CAP 에서 P 는 필수 → CP 나 AP 나 의 선택이고 , W·R 숫자가 그 손잡이

7 장 Snowflake 는 64 비트를 의미 단위로 쪼개 시간 정렬과 조율 없는 생성을 동시에 얻음

discussion.md

토의 질문

- 1 장바구니 사본이 3 벌 (N=3) 입니다 . 담기는 초당 수만 건이고 , 결제 직전에 상품이 안 보이면 매출로 직결됩니다 . W 와 R 을 각각 얼마로 잡으시겠습니까 ?
- 2 충돌이 난 장바구니를 합집합으로 합치면 , 사용자가 삭제한 상품이 되살아나지 않을까요 ?
- 3 NTP 보정으로 시계가 2 초 뒤로 가면 , 이미 ID 를 발급한 시간대를 다시 지나갑니다 . 그대로 발급해 중복을 감수할까요 , 2 초간 발급을 멈출까요 ?